EmuTOS on the Mini-68K: port plan

Oct 11, 2026 · @Dan Werner

The plan is to bring EmuTOS up on the serial console first, then run GEM on the VERA card through a RAM shadow framebuffer, and finally replace the shadow copy with a native VERA driver. Hardware and EmuTOS details here are from memory and are approximate until checked against the board, the VERA reference and the EmuTOS source.

Hardware assumptions

The plan assumes the board below; each row marked "confirm" changes the work if it is wrong.

Item	Assumed	Why it matters
CPU	68008, about 8 MHz (confirm clock and package)	The 48-pin part has a 1 MB address space and only interrupt levels 2, 5 and 7; the 52-pin part has 4 MB and all seven levels.
RAM	About 2 MB, starting at address 0 and writable (confirm)	EmuTOS needs writable exception vectors and system variables in low memory.
ROM	Flash large enough for a 256 KB image (confirm size and address)	If it is smaller, EmuTOS must be loaded from disk into RAM.
MMU	None	Fine for EmuTOS; rules out trapping screen writes.
Serial	16550 UART on the MF/PIC card	Console for Phase 1; needs a new EmuTOS driver.
Interrupts and timer	NS32202 on the MF/PIC card (confirm)	Source of the 200 Hz system tick and the route for VERA's vsync interrupt.

Item	Assumed	Why it matters
Disk	IDE through an 8255 (PPIDE) on the MF/PIC card (confirm)	EmuTOS's IDE driver expects memory-mapped registers, so the low-level access must be rewritten.
Clock	Battery RTC on the MF/PIC card (confirm chip)	Optional; file timestamps only.
Video	VERA, port accessed, working under existing software	GEM display from Phase 2 on.
Keyboard and mouse	Not yet decided	See Phase 3.
Bus errors	Unknown whether unmapped addresses raise a bus error	EmuTOS probes for hardware this way; without it the probes must be compiled out.

Phases at a glance

Each phase ends in something you can use, and nothing above the BIOS layer changes when the display path is swapped in Phase 4.

Phase	Deliverable	Exit test	Experiments first
0. Groundwork	Cross toolchain, EmuTOS fork, fast load-and-test loop	A stub image loads and prints over serial	E1, E2
1. Serial EmuTOS	EmuTOS boots to the EmuCON shell with a FAT disk	Sozobon C compiles and runs hello-world on the board	E3, E4, E5
2. Shadow framebuffer	GEM desktop displayed on VERA	EmuDesk opens windows and menus; full redraw under about 0.3 s	E6, E7, E8, E9

Phase	Deliverable	Exit test	Experiments first
3. Input	Keyboard and mouse driving GEM	A GEM application is usable without the serial terminal	E10
4. Native driver	VDI and console draw to VERA directly	Shadow buffer removed in the default build; color mode works	E11, E12, E13

Phase 0: groundwork

The goal is a loop where a rebuilt image is running on the board within a minute or two, without reflashing.

- [ ] Install the `m68k-atari-mint` GCC cross toolchain on a PC and build an unmodified EmuTOS target to prove the toolchain.
- [ ] Fork EmuTOS and add a new machine define for the Mini-68K, modelled on how the Amiga and ColdFire board ports are separated from the Atari code.
- [ ] Write down the board's memory map: RAM range, ROM range, I/O port addresses for the UART, interrupt controller, PPIDE, RTC and VERA.
- [ ] Decide where EmuTOS lives. Preferred during development: a RAM-loadable image started from the existing monitor or CP/M-68K (E1). Later: flash.
- [ ] Build a stub that sets the stack, initialises the UART and prints a banner, using the same link address and load path the real image will use.
- [ ] Settle interrupt levels for the system tick, the UART and VERA vsync, given the levels your 68008 package provides.
- [ ] Run the bus-error test (E2) and record the result for Phase 1.

Phase 1: EmuTOS on the serial console

This phase is a conventional BIOS port: four small drivers and a configuration with all Atari hardware switched off.

Configuration

- [ ] Disable every Atari-specific device in the build configuration: MFP, keyboard ACIA, sound chip, video shifter, floppy, ACSI, blitter, NVRAM.

- [ ] Enable EmuTOS's serial-console option so the shell and text programs use the UART.
- [ ] Compile out hardware probes if E2 shows the board does not raise bus errors.
- [ ] Replace memory sizing with a fixed or simple test-based RAM size.

Drivers to write

- [ ] 16550 UART: character in, character out, status, and receive interrupt into EmuTOS's serial buffer.
- [ ] System tick: a 200 Hz interrupt from the interrupt controller's counter, wired to EmuTOS's timer handler (E3).
- [ ] Interrupt glue: vectors and acknowledge sequence for the NS32202, and an interim 50 or 60 Hz vertical-blank tick derived from the 200 Hz timer until VERA supplies a real one.
- [ ] PPIDE block driver: sector read and write through the 8255, plugged in beneath EmuTOS's existing disk and partition layer (E4).
- [ ] RTC read and set (optional; can wait).

Bring-up order

1. Banner and early debug output over the UART.
2. Tick interrupt running; EmuTOS reaches the EmuCON prompt with no disk.
3. Disk: mount a FAT16 partition, list files, run a `.TOS` program.
4. Tools: copy Sozobon C, a make and an editor to the disk and build something on the board.

Decisions

- Terminal emulation: use a VT52 terminal, or enable the console's ANSI translation if it works well enough (E5).
- Disk layout: a separate CompactFlash for EmuTOS is simplest. Sharing one disk with CP/M-68K needs a partition table both systems accept.